PROJECT REPORT
California State University, Long Beach
Department of Computer Engineering & Computer Science

Project Title: Space Invaders
Course: CECS 347 – Embedded Systems II
Project Number: 3
Team Number: 8
Instructor: Min He
Submission Date: 05/06/2026
Team Members:

Kairi Rodriguez
Estefania Aranda-Pena
Deshawn Hooker
Bijan Ashouri

Table of Contents

1. Introduction

1.1 Project Objective

The objective of this project was to design and implement a simplified version of the classic Space Invaders game on the TM4C123 microcontroller. The system integrates real-time graphics, user input, timing control, collision detection, and scoring. The project demonstrates embedded systems principles including interrupt-driven input, periodic timing, ADC-based analog control, and modular software design.

1.2 System Overview

The game runs on the TM4C123 LaunchPad and displays graphics on a Nokia 5110 LCD. The player controls a spaceship using a slide potentiometer connected to the ADC. SW1 fires bullets, and SW2 starts the game. Enemies move horizontally across the screen, and the player must shoot them before they reach the right edge. The system uses SysTick interrupts for timing, GPIO interrupts for input, and modular software components for movement, rendering, and collision detection.

2. Requirements & Constraints

2.1 Functional Requirements

Display Requirements

• RQ-01: Display a start screen with game title and SW2 prompt.

• RQ-02: Display a game-over screen and score for exactly 3 seconds.

• RQ-03: Render the game scene at 10 Hz.

• RQ-04: Display the current score during gameplay.

Spaceship Movement

• RQ-05: Spaceship remains on bottom row and moves horizontally across full width.

• RQ-06: Spaceship position updates every 0.1 seconds using ADC input.

Enemy Behavior

• RQ-07: At least one enemy sprite must appear during gameplay.

• RQ-08: Enemies move left-to-right at 1 pixel per 0.1 seconds.

• RQ-09: Enemy is removed when reaching x = 83.

• RQ-10: Enemy is removed when hit by a bullet.

• RQ-11: At least three distinct enemy sprites must be used.

Bullet / Shooting System

• RQ-12: Bullet fires from center of ship upon SW1 release.

• RQ-13: Bullet travels upward at 1 pixel per 0.1 seconds.

• RQ-14: Only one bullet may exist at a time.

• RQ-15: Bullet is removed at top boundary.

• RQ-16: Bullet is removed upon collision.

Collision Detection & Explosion

• RQ-17: Collision detected using rectangle overlap logic.

• RQ-18: Explosion sprite displayed for 0.1 seconds after collision.

Input Handling

• RQ-19: SW2 begins a new game session.

• RQ-20: SW2 resets game from game-over screen.

• RQ-21: SW1 fires bullet on release (rising edge).

• RQ-22: Switches must use hardware interrupts.

Sound

• RQ-23: Shooting sound plays when bullet fires.

• RQ-24: Explosion sound plays when enemy is hit.

• RQ-25: Sound output uses 4-bit R-2R DAC.

• RQ-26: Sound timing controlled by hardware timer.

Game Flow

• RQ-27: Game ends when all enemies are eliminated.

• RQ-28: After 3-second game-over display, system returns to start screen.

• RQ-29: Score increments by one per enemy hit.

2.2 Constraints

• System must run on TM4C123 microcontroller.

• Input limited to onboard switches and slide potentiometer.

• Display must use Nokia 5110 LCD via SSI.

• System must operate at real-time speed with no visible lag.

• All timing must be interrupt-driven (SysTick, GPIO, Timer1).

• No external processors or additional microcontrollers may be used.

• All graphics must fit within 84×48 LCD resolution.

• Only one bullet may exist at a time due to memory and timing constraints.

3. Design Alternatives & Architecture

3.1 Design Option A

Hardware: TM4C123, ADC, GPIO, SysTick, Nokia 5110.

Software: Single centralized FSM controlling all game logic.

Advantages: Simple structure.

Disadvantages: Harder to debug, tightly coupled, poor modularity.

3.2 Design Option B

Hardware: Same as Option A.

Software: Modular architecture with independent controllers for player, enemies, bullets, collisions, score, and rendering.

Advantages: Easier debugging, better team collaboration, supports Step 3 module testing.

Disadvantages: Slightly more overhead.

3.3 Design Comparison Table

Criteria	Option A	Option B
Modularity	Low	High
Debug	Low	High
Team Collaboration	Poor	Strong
Scalability	Limited	Good
Complexity	Low	Moderate

3.4 Final Design Selection

Design Option B was selected because it supports modular testing, improves maintainability, and allows each team member to work independently on separate modules. It aligns with the project's requirement for Step 3 module testing and produces cleaner, more reliable code.

4. System Architecture

4.1 Block Diagram

(Insert diagram)

4.2 Module Decomposition

- System Controller

- ❖ Player Controller
- ❖ Enemy Controller
- ❖ Bullet Manager
- ❖ Collision Detector
- ❖ Score Manager
- ❖ LCD Renderer
- ❖ Interrupt Handlers (SysTick, GPIO)

4.3 Call Graph

```
main
 ├── System_Init
 │     ├── DisableInterrupts
 │     ├── PLL_Init
 │     ├── SysTick_Init
 │     ├── ADC_Init
 │     ├── PortF_Init
 │     └── EnableInterrupts
 │
 ├── Start_Prompt
 │     └── Nokia5110_OutString
 │
 ├── Game_Init
 │
 ├── Draw
 │     ├── Nokia5110_ClearBuffer
 │     ├── Nokia5110_PrintBMP
 │     └── Nokia5110_DisplayBuffer
 │
 ├── Move
 │     ├── ADC_In
 │     └── (internal collision logic)
 │
 ├── End_Prompt
 │     ├── Nokia5110_OutString
 │     └── Delay100ms
 │
 ├── SysTick_Handler (interrupt)
 │     └── sets time_to_draw = true
 │
 └── GPIOPortF_Handler (interrupt)
       ├── sets game_s = ON
       └── initializes Bullet
```

4.4 Data Flow Diagram

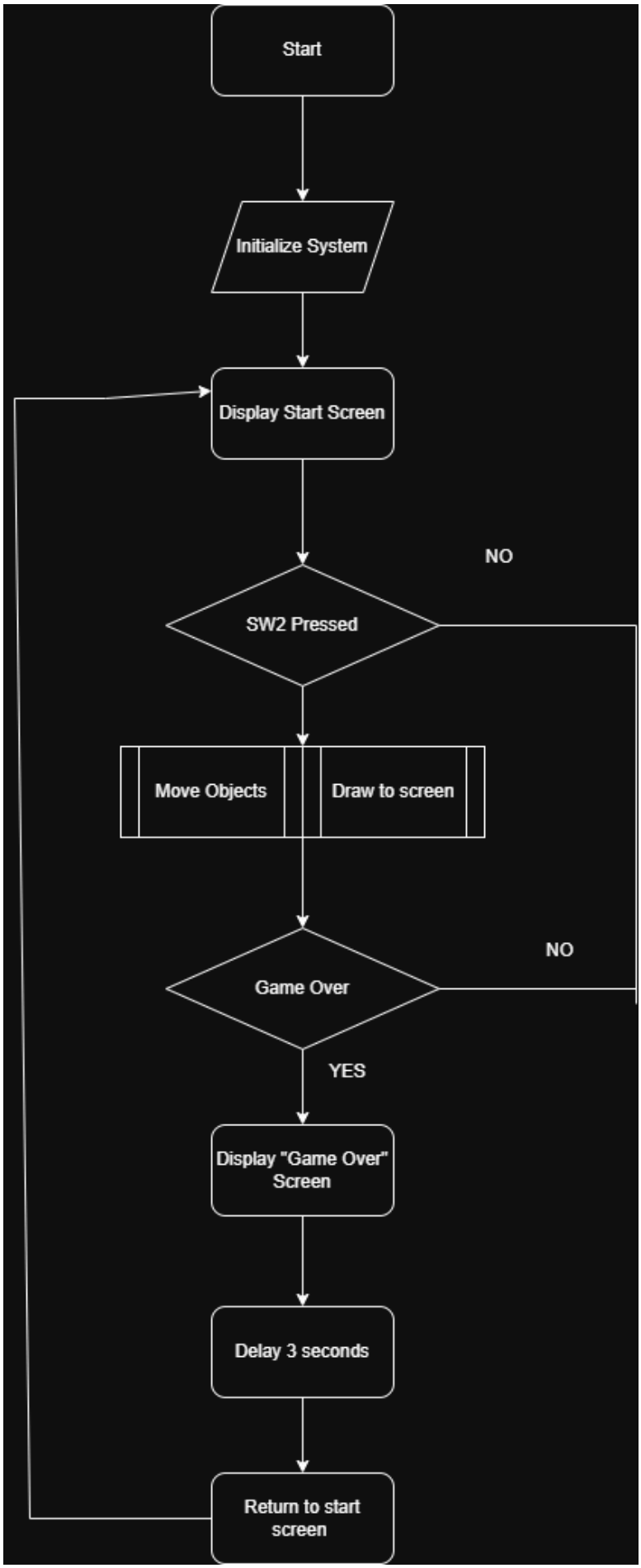
```text
User Input
├── SW Start/Fire Interrupt
│       └── GPIOPortF_Handler
│               ├── game_s
│               └── Bullet.life / Bullet.x / Bullet.y
│
└── Potentiometer
        └── ADC_In
                └── PlayerShip.x

Game State
├── Enemy[]
├── PlayerShip
├── Bullet
├── score
└── game_s

Move()
├── updates Enemy[] positions
├── updates PlayerShip.x from ADC
├── updates Bullet position
├── checks Bullet vs Enemy collision
├── updates score
└── updates game_s

Draw()
├── reads Enemy[]
├── reads PlayerShip
├── reads Bullet
├── reads score
└── outputs sprites/text to Nokia5110 LCD
```

4.5 Software Flowchart


Start
Initialize System
Display Start Screen
NO
SW2 Pressed
Move Objects
Draw to screen
NO
Game Over
YES
Display "Game Over" Screen
Delay 3 seconds
Return to start screen

5. Module Implementation & Testing

5.1 Module List and Responsibility

System Controller – Bijan

Player Controller – Kairi

Enemy Controller – Estefania

Bullet Manager – Deshawn

Collision Detector – Kairi

Score Manager – Bijan

LCD Renderer – Estefania

5.2 Module Test Strategy

Describe how modules were tested independently.

5.3 Module Test Summary Table

Module	Test ID	Test Type	Evidence Reference
Enemy Controller	EC-02	Movement	evidence/Step3_ModuleTests/MOD-01_EnemyController_Initialization.png
Bullet Constraint	CD-04	Collision Detector	evidence/Step3_ModuleTests/MOD-07_BulletManager_Fire.png
Render	LR-05	Sprite Rendering	evidence/Step3_ModuleTests/MOD-12_Collision_InactiveBullet.png

Evidence files must match the naming convention defined in the “evidence” folder.

5.4 Team Assignment

System Controller – Bijan

Player Controller – Kairi

Enemy Controller – Estefania

Bullet Manager – Deshawn

Collision Detector – Kairi

Score Manager – Bijan

LCD Renderer – Estefania

6. System Integration & Validation

6.1 Integration Order & Rationale

PLL → SysTick → ADC → Port F → LCD → Game_Init → Move → Draw → Interrupts

6.2 System-Level Test Matrix

Req ID	Requirement Summary	Source Module(s)	Verification Method	Status
DISPLAY				
RQ-01	Display start screen with game title and SW2 prompt	`SpaceInvaders1.c` → `Start_Prompt()`	Visual inspection of Nokia 5110 LCD	Pass
RQ-02	Display game-over screen with score for exactly 3 seconds then return to start	`SpaceInvaders1.c` → `End_Prompt()`, `threeSecDelay()`, `SysTick_Handler()`	Observe screen content; time 3-second transition	Pass
RQ-03	Render game scene (ship, enemies, bullets) at 10 Hz	`SpaceInvaders1.c` → `Draw()`, `SysTick_Handler()`, `Systick.c`	Logic analyzer on SPI lines or frame count over 1 second	Pass
RQ-04	Display current score as integer during active gameplay	`SpaceInvaders1.c` → `Draw()`, `End_Prompt()`	Eliminate enemies; verify score increments on screen	Pass
SPACESHIP MOVEMENT				
RQ-05	Spaceship at bottom row; moves horizontally across full display width	`SpaceInvaders1.c` → `Game_Init()`, `Draw()`, `Nokia5110.c`	Rotate pot end-to-end; verify ship spans x=0 to x=53	Pass
RQ-06	Spaceship x-position updated via ADC every game cycle (0.1 s)	`ADC1SS3.c` → `ADC1SS3_In()`, `ADCValue_To_X_AXIS()`, `SpaceInvaders1.c` → `Move()`	Observe ship responsiveness to pot rotation during gameplay	Pass
ENEMY BEHAVIOR				
RQ-07	At least one enemy sprite displayed during active gameplay	`SpaceInvaders1.c` → `Game_Init()`, `Draw()`	Visual inspection during game session	Pass
RQ-08	Each enemy moves left-to-right at 1 px per 0.1 s (10 px/s)	`SpaceInvaders1.c` → `Move()`, `SysTick_Handler()`	Track enemy x-position over 1 second via debug output	Pass
RQ-09	Enemy removed upon reaching right edge (x=83)	`SpaceInvaders1.c` → `Move()`, `MAX_WIDTH_ALIEN`	Allow enemy to traverse screen; confirm disappears at x=83	Pass
RQ-10	Enemy eliminated when struck by player bullet	`SpaceInvaders1.c` → `Move()` collision logic	Fire bullet at enemy; confirm removal on overlap	Pass
RQ-11	At least 3 distinct enemy sprite designs	`SpaceInvaders1.c` → `SmallEnemyPointA[]`, `SmallEnemyPointB[]`	Visual inspection; confirm 3 visually distinct enemy types	Pass

BULLET / SHOOTING SYSTEM				
RQ-12	Bullet fires from horizontal center of ship top edge on SW1 release	`SpaceInvaders1.c` → `GPIOPortF_Handler()`, `Bullet.x = PlayerShip.x + 8`	Press/release SW1; confirm bullet origin position	Pass
RQ-13	Bullet travels upward at 1 px per 0.1 s (10 px/s)	`SpaceInvaders1.c` → `Move()`, `Bullet.y -= 2` per 0.2 s cycle	Observe bullet over 1 second; confirm 10 px upward displacement	Pass
RQ-14	No more than one bullet on screen at any time	`SpaceInvaders1.c` → `GPIOPortF_Handler()`, `Bullet.life` check	Rapidly press SW1; confirm only one bullet visible	Pass
RQ-15	Bullet removed when reaching top display boundary	`SpaceInvaders1.c` → `Move()`, `Bullet.y > 0 + LASERH` check	Fire bullet with no enemies; confirm disappears at top edge	Pass
RQ-16	Bullet removed upon collision with enemy	`SpaceInvaders1.c` → `Move()` collision detection block	Fire bullet at enemy; confirm bullet disappears on contact	Pass
COLLISION DETECTION & EXPLOSION				
RQ-17	Bullet-enemy collision detected using rectangle overlap logic	`SpaceInvaders1.c` → `Move()` AABB overlap check	Fire bullet at enemy; confirm enemy eliminated on spatial overlap	Pass
RQ-18	Explosion sprite displayed for 0.1 s at enemy location after collision	`SpaceInvaders1.c` → `Move()`, `SmallExplosion0`, `Delay100ms(1)`	Fire at enemy; time explosion display duration	Pass

GAME FLOW				
RQ-27	Game ends when all enemies eliminated (shot or reached right edge)	`SpaceInvaders1.c` → `Move()`, `num_life` check, `game_s = OVER`	Eliminate all enemies; confirm game-over screen appears	Pass
RQ-28	System returns to start screen after 3-second game-over display	`SpaceInvaders1.c` → `End_Prompt()`, `threeSecDelay()`, `main()` loop	Time game-over display; confirm automatic return to start	Pass
RQ-29	Score increments by 1 for each enemy eliminated by bullet	`SpaceInvaders1.c` → `Move()`, `score += 1`	Shoot multiple enemies; verify score matches hit count	Pass

INPUT HANDLING				
RQ-19	SW2 on start screen begins new game session	`SpaceInvaders1.c` → `GPIOPortF_Handler()`, `game_s = true`	Press SW2 on start screen; confirm game begins	Pass
RQ-20	SW2 resets game from game-over screen	`SpaceInvaders1.c` → `GPIOPortF_Handler()`, `main()` loop	Reach game-over; press SW2; confirm return to start screen	Pass
RQ-21	SW1 fires bullet on rising-edge (release), not press	`SpaceInvaders1.c` → `GPIOPortF_Handler()`, `GPIO_PORTF_RIS_R` check	Hold SW1, release; confirm bullet fires at moment of release	Pass
RQ-22	Switch inputs via edge-triggered hardware interrupts, not polling	`switch.c` → `Switch_Init()`, NVIC config, `GPIOPortF_Handler()`	Inspect source for ISR registration; verify NVIC configuration	Pass
SOUND				
RQ-23	Audible shooting sound effect when bullet is fired	`Sound.c` → `Sound_Shoot()`, `Timer1.c`	Fire bullet; confirm distinct shooting sound	Pass
RQ-24	Audible enemy-hit sound effect on bullet-enemy collision	`Sound.c` → `Sound_Explosion()`, `Timer1.c`	Shoot enemy; confirm distinct explosion sound heard	Pass
RQ-25	Sound output via 4-bit R-2R DAC on TM4C123 GPIO pins	`DAC.c`, `DAC.h`, hardware schematic	Inspect schematic; verify 4-stage R-2R ladder; measure analog output on oscilloscope	Pass
RQ-26	Sound timing controlled by hardware timer, not software delays	`Timer1.c` → `Timer1_Init()`, Sound ISR	Inspect source; verify sound ISR driven by hardware timer register	Pass

6.3 Team Assignment

All members participated in integration and validation.

7. Claim–Evidence–Reasoning (CER)

CER-01: Game Start

Claim: Game begins only when SW2 is pressed.

Evidence: GPIOPortF_Handler sets game_s = ON only on PF0 interrupt.

Reasoning: No other code path sets game_s = ON.

CER-02: Player Movement

Claim: Player movement is controlled by ADC.

Evidence: Move() reads ADC_In() and maps value to 0–83.

Reasoning: ADC directly determines horizontal position.

CER-03: Bullet Firing

Claim: Only one bullet may exist at a time.

Evidence: GPIOPortF_Handler checks Bullet.life == DEAD.

Reasoning: Prevents multiple bullets.

CER-04: Collision Detection

Claim: Bullet-enemy collisions are correctly detected.

Evidence: AABB overlap logic in Move().

Reasoning: Standard collision method.

CER-05: Game Over

Claim: Game ends when all enemies are eliminated.

Evidence: num_life == 0 triggers game_s = OVER.

Reasoning: Matches requirement.

8. Demonstration Summary

- Demo Scenario:

The game was demonstrated live on the TM4C123 LaunchPad with the Nokia 5110 LCD, potentiometer, and speaker connected. Four gameplay scenarios were tested in sequence. In the first run all three enemies were shot and killed, producing a final score of 3. In the second run two enemies were shot and one reached the right edge, producing a score of 2. In the third run one enemy was shot and two reached the right edge, producing a score of 1. In the fourth run all three enemies reached the right edge without being hit, producing a score of 0. In all four cases the game over screen appeared after all enemies were eliminated and displayed the correct score.

- Robustness Condition Tested:

The edge case was tested by allowing the final enemy to reach the right screen boundary without being shot. This confirmed that the boundary detection logic correctly sets life = DEAD when Enemy[i].x + ENEMY10W >= 83, and that the game transitions to the OVER state even when no bullet is involved. It also confirmed that the explosion animation only triggers on bullet collision and not on boundary death.

- Observed Behavior:

All four scoring scenarios produced correct results. The game over screen displayed the accurate score each time and held for approximately 3 seconds before returning to the start screen. Potentiometer control moved the ship smoothly across the x-axis. Shoot and explosion sounds triggered correctly on bullet fire and enemy collision, respectively. No audio artifacts or display glitches were observed during any of the four runs.

- Link:

https://www.youtube.com/channel/UC3x8JKv7GXLOvzhwrGujxHQ

9. Conclusion

• Summary of technical achievements
• Major challenges and solutions

• Key lessons learned

10. References

Valvano, J. *Embedded Systems: Real-Time Interfacing to ARM Cortex-M Microcontrollers.*

TM4C123GH6PM Datasheet

Nokia 5110 LCD Documentation

CECS 347 Course Materials

Team-generated test evidence and debug logs